

AI BUILDERS IRAN TEAM

Warehouse Safety Monitoring System

Backend Engineering Review Report

FastAPI Service, Gradio Client & Containerization Layer

PREPARED FOR

LLM & Backend Sub-Teams

SCOPE

API/, UI/ (backend delivery)

STATUS

Technical Review, Pre-Integration

DATE

August 6, 2026

1. Executive Summary

This document reviews the backend deliverables submitted for the Warehouse Safety Monitoring System: a two-service application consisting of a **FastAPI orchestration layer** (API/) and a **Gradio front-end client** (UI/). The backend does not re-implement detection, tracking, rule evaluation, or language generation; instead it acts as the **integration and orchestration layer** that wires together three existing modules — the computer-vision pipeline (YOLO/mains.py), the rule engine, and the local LLM service (LLM/llm_client.py, Qwen2.5) — behind a single HTTP contract.

Overall, the submitted code is well-structured, clearly commented, and demonstrates good separation of concerns: the UI never imports model code directly, the API loads heavy objects once at startup, and failure of any one component (missing GPU, missing YOLO weights) degrades gracefully rather than crashing the service. A deliberate and well-documented defensive normalization layer bridges a real schema mismatch between the rule engine's alert output and the prompt generator's expected input — this is the single most notable piece of engineering judgment in the submission and is flagged as a cross-team follow-up item below (see §4.1).

Six items are flagged for attention before this is considered integration-ready, the two most important being: **(1)** the shared alert schema issue between the rule engine and the LLM prompt layer, currently papered over by an API-side normalization shim, and **(2)** an open CORS policy with no authentication on the API. Two medium-severity items — a missing docker-compose.yml and unpinned dependencies on a bleeding-edge Python 3.14 base image — plus two low-severity cleanup items round out the list. None of these block a code walkthrough, but the two high-severity items should be resolved before a staging deployment (see §4 for the full list).

At a Glance

Item	Summary
Services delivered	2 — FastAPI backend (API/), Gradio client (UI/)
Primary endpoints	GET /health, POST /analyze
External dependencies wired	YOLO/mains.py, LLM/llm_client.py, LLM/prompt.py
Containerization	Per-service Dockerfiles present; compose file missing
GPU requirement	Hard CUDA requirement for LLM inference; API stays up in degraded mode without it
Open findings	6 (see §4) — 2 High, 2 Medium, 2 Low

2. Architecture Overview

The system follows a **thin-client / heavy-service** pattern. The Gradio UI never touches model code — it communicates with the API exclusively over HTTP, which allows the two services to be deployed, scaled, and versioned independently (e.g. a CPU-only UI container talking to a GPU-backed API container).

2.1 Request Flow

- User uploads a video and selects a report language (fa/en) and style (summary / detailed / json), or supplies a free-text custom prompt, via the Gradio interface (UI/app.py).
- The UI posts the video and form parameters to POST /analyze on the API service.
- The API persists the upload to a temp file and passes it to `SafetyVideoPipeline.process_video()`, which runs YOLO detection, tracking, and rule evaluation and returns `video_info`, `tracking_summary`, and `alerts_report`.
- The raw alerts are normalized (see §4.1) and passed into either `HSEPromptGenerator` (ready-made styles) or `AlertsSummarizer` (custom-prompt mode) to build the final LLM prompt.
- The prompt is sent to the locally hosted Qwen2.5 model via `LLMModel.generate()`; the generated HSE report text is returned to the UI along with the underlying structured data.

2.2 Component Map

Module	Type	Responsibility
UI/app.py	Gradio Blocks app	Video upload form, language/style/custom-prompt controls, results display, backend health check button.
API/app.py	FastAPI service	Owns the /health and /analyze contract; loads the YOLO pipeline and LLM once at startup via a lifespan context manager; normalizes alert data; builds prompts; calls the LLM; returns a typed Pydantic response.
YOLO/mains.py *	Imported, not modified	SafetyVideoPipeline — detection, tracking, rule engine (external module, referenced only).
LLM/lm_client.py *	Imported, not modified	LLMModel — local Qwen2.5 inference; requires CUDA (external module, referenced only).
LLM/prompt.py *	Imported, not modified	HSEPromptGenerator, AlertsSummarizer — prompt templates in fa/en (external module, referenced only).

* Not part of this backend delivery; listed for context since API/app.py is the integration point.

2.3 API Contract

Endpoint	Key Parameters	Behavior
GET /health	—	Liveness probe. Reports whether the YOLO pipeline and LLM loaded successfully, with error detail if not.
POST /analyze	video (file), language (fa/en), mode (summary/detailed/json), custom_prompt (optional), max_new_tokens	Runs the full pipeline and returns video_info, tracking_summary, alerts_report, report_text, and the exact prompt used.

3. Notable Strengths

- ✓ **Clean service boundary**
The UI communicates with the API purely over HTTP and never imports model code. This makes the two services independently deployable and testable, and keeps the lightweight UI image free of torch/ultralytics dependencies.
- ✓ **Startup-once resource loading**
The YOLO pipeline and LLM are instantiated once in a FastAPI lifespan handler rather than per-request, avoiding repeated model-load latency and VRAM churn.
- ✓ **Graceful degradation without a GPU**
If CUDA is unavailable, LLM initialization failure is logged as a warning (not an error) and the API stays up — vision-only features keep working, and callers get a clear 503 with the underlying error message only when they hit the LLM-dependent path.
- ✓ **Explicit, documented data-contract fix**
The `_normalize_alerts_for_summarizer()` helper is a deliberate, well-commented fix for a real mismatch between the rule engine's alert schema and the prompt generator's assumptions, applied without modifying either upstream module (see §4.1 for the cross-team implication).
- ✓ **Configurable via environment, not hard-coded**
Model paths, confidence/IOU thresholds, upload limits, and token-generation bounds are all environment-overridable, which will simplify moving from a dev machine to shared GPU infrastructure.
- ✓ **Sensible request-level validation**
Language, mode, and `max_new_tokens` are all validated against explicit allow-lists/bounds before any expensive work (video processing, LLM generation) begins, returning 422 early rather than failing deep in the pipeline.

4. Findings & Recommendations

Findings are ordered by severity. Severity reflects risk if the current code were deployed to a shared/staging environment as-is, not correctness of the code for local development.

4.1. Alert schema mismatch between rule engine and prompt generator

HIGH

Per the in-code documentation, `AlertsSummarizer._compute_track_summary()` unconditionally reads `track_id`, `class_name`, and `speed` on every alert, but the rule engine only attaches `class_name/speed` to `high_speed` alerts, and crowd-detection alerts carry no `track_id` at all. Without the API's defensive normalization step, any realistic alert set would crash prompt generation with a `KeyError`. The API-side fix is a reasonable short-term patch, but it means the true fix belongs upstream, in the rule engine / prompt-generator contract, where the two computer-vision and LLM sub-teams will need to agree on a single alert schema.

Recommendation: Treat `_normalize_alerts_for_summarizer()` as a temporary shim. Raise this with the CV and LLM sub-teams as a shared-schema task so the rule engine emits a consistent alert shape (`track_id/class_name/speed` on every alert type) and the normalization layer can eventually be retired.

4.2. Open CORS policy and no authentication on the API

HIGH

`CORSMiddleware` is configured with `allow_origins=["*"]`, `allow_credentials=True`, and all methods/headers open. Combined with no authentication on `/analyze`, any origin that can reach the API can upload video and consume GPU inference time.

Recommendation: Acceptable for local development; before any shared or staging deployment, restrict `allow_origins` to the known UI origin(s) and add at minimum an API-key or internal-network restriction on `/analyze`, since it is compute-expensive.

4.3. docker-compose.yml is referenced but not included

MEDIUM

Comments in `API/Dockerfile`, `UI/Dockerfile`, and `UI/app.py` all describe compose-driven behavior (build context = repo root, `API_URL=http://api:8000` service networking, GPU passthrough via `nvidia-container-toolkit`), but no `docker-compose.yml` was present in the delivered archive. The two Dockerfiles cannot be run standalone as documented — the API image expects a build context containing sibling `YOLO/` and `LLM/` directories that only a root-context build would provide.

Recommendation: Include the compose file (or a build/run script) in the next delivery so the team can reproduce the intended multi-container setup, including the GPU reservation for the API service.

4.4. Unpinned dependencies on a bleeding-edge base image

MEDIUM

Both `requirements.txt` files list packages with no version pins (`fastapi`, `torch`, `ultralytics`, `gradio`, etc.), and both Dockerfiles build on `python:3.14-slim`. Python 3.14 is very recent; `torch/ultralytics` wheel availability for it is not guaranteed at build time, and unpinned installs mean the image is not reproducible across builds.

Recommendation: Pin at least the major/minor versions of GPU-critical packages (`torch`, `transformers`, `ultralytics`) and verify prebuilt wheels exist for the chosen Python version — or step down to a more established Python version (e.g. 3.11/3.12) for the API image until 3.14 wheel coverage is confirmed.

4.5. Empty root-level Dockerfile

LOW

An empty Dockerfile exists at the project root alongside the populated `API/Dockerfile` and `UI/Dockerfile`. It appears to be an unused leftover rather than an intentional multi-stage or aggregator build.

Recommendation: Remove it if unused, or repurpose it as the compose-context anchor / documentation entry point if that was the intent — either way, avoid ambiguity for the next person building the image.

4.6. No automated tests included

LOW

The delivered archive contains application code and Docker configuration only; no test suite (unit tests for the normalization helper, integration test for `/analyze` with a mocked pipeline/LLM, etc.) was included.

Recommendation: Given `_normalize_alerts_for_summarizer()` encodes an important cross-team assumption about alert shape, it is a good first candidate for a small unit test to catch regressions if the rule engine's output format changes.

5. Configuration Reference

Environment variables recognized by the two services, for use when preparing the missing compose file (§4.3).

Variable	Service	Default	Purpose
YOLO_MODEL_PATH	API	YOLO/models/best.onnx	Path to the ONNX detection weights.
LLM_MODEL_PATH	API	LLM/model	Path to the local Qwen2.5 model directory.
YOLO_CONF_THRESHOLD	API	0.3	Detection confidence threshold.
YOLO_IOU_THRESHOLD	API	0.45	NMS IoU threshold.
MAX_UPLOAD_MB	API	500	Maximum accepted video upload size.
DEFAULT_MIN_MAX_NEW_TOKENS	API & UI	300 / 16 / 2048	LLM generation length bounds; must be kept in sync between services (UI mirrors these to render its slider without an extra API call).
LOG_LEVEL	API	INFO	Python logging level.
API_URL	UI	http://api:8000	Backend base URL the Gradio client calls; the compose-service-name default confirms compose is the intended deployment path.
REQUEST_TIMEOUT_SECONDS	UI	1800	Client-side timeout for the /analyze call — generous, appropriate for large video + LLM generation.
GRADIO_PORT	UI	7860	Port the Gradio app binds to.

Hardware & Runtime Requirements

- The API image requires an **NVIDIA GPU + nvidia-container-toolkit** on the host at runtime, since LLM/llm_client.py hard-requires CUDA; the API remains reachable without one, but every request touching /analyze will return 503 for the LLM step.
- The UI image has no GPU dependency and intentionally excludes torch/ultralytics, keeping it lightweight and independently scalable.
- System packages libgl1, libglv2.0-0, and ffmpeg are installed in the API image to support OpenCV and video decoding.

6. Summary of Action Items

Priority	Action	Owner
HIGH	Coordinate with CV/LLM sub-teams to unify the alert schema; retire the API-side normalization shim once fixed.	Backend team
HIGH	Lock down CORS to known origins and add access control to /analyze before any shared deployment.	Backend team
MED	Add the missing docker-compose.yml (with GPU reservation for the API service).	Backend team
MED	Pin GPU-critical package versions; confirm wheel availability for the chosen PyTorch base image.	Backend team
LOW	Remove or repurpose the empty root Dockerfile.	Backend team
LOW	Add a unit test around the alert-normalization helper.	Backend team

Prepared as an internal engineering review of the backend deliverable for the Warehouse Safety Monitoring System. Scope was limited to the files in the submitted archive (API/, UI/, and associated Docker configuration); the YOLO, rule-engine, and LLM modules were reviewed only at the integration boundary, not in full.